Social Recommendation with Graph Neural Networks Integrating Transformer and Self-Supervised Learning

Xingyao Yang[1*], Shuai Ma[1], Jiong Yu[1], Guangchao Li[1], Zheng Qi[1]

[1]School of Software, Xinjiang University, Northwest Road, Urumqi, 830091, Xinjiang Uygur Autonomous Region, China.

*Corresponding author(s). E-mail(s): yangxy@xju.edu.cn;

Abstract

Social recommendation utilizes social information to address data sparsity and cold start issues. While Graph Neural Networks (GNNs) are naturally suited for this graph-structured task, they face challenges in modeling complex topologies and effectively integrating user-item interactions with social relationships. To overcome these limitations, we propose SSLGTSR, a social recommendation model that integrates the Graph Transformer with self-supervised learning. SSLGTSR first injects global topological information using three distinct positional encodings to understand the graph structure. It then expands the model's receptive field by alternating between Transformer and GNN layers. Furthermore, we introduce a novel self-supervised learning task to align user embeddings from interaction and social views, ensuring social awareness consistency. SSLGTSR achieves superior Recall and NDCG on three benchmark datasets: LastFM, Ciao, and Douban-Book. Compared to the suboptimal baseline, it improves Recall by 2.60%, 3.55%, and 4.35%, and NDCG by 3.38%, 3.55%, and 3.63% respectively, demonstrating the model's effectiveness.

Keywords: Recommender System, Graph Neural Networks, Graph Transformer, Self-supervised learning, Social Recommendation

1 Introduction

In social networks, users' preferences are influenced by their social relationships or they establish social relationships because of similar preferences [1, 2]. Social recommendation leverages these interactions to enrich user representation, address data sparsity and cold start problems in user-item interaction networks, and enhance recommendation performance [3–6]. Consequently, social recommendation has garnered significant attention from researchers and is a current research hotspot [7].

Since user-item interactions and user social relationships can be modeled as graph structures, many methods now apply GNNs to social recommendation [8–11]. In user-item interaction bipartite graphs and user social graphs, information propagates iteratively, influencing user characteristics through both first-order and higher-order neighbors. Capturing the influence of these higher-order neighbors is crucial for improving recommendation system performance [12–16]. Previous works, such as HOSR [17] and SocialLGN [18], model users using multiple iterations of graph convolution operations to obtain high-order neighbor embeddings. However, shallow GNNs with limited receptive fields can only aggregate partial neighborhood information, while deeper GNNs may over-aggregate, making the embedding layers become too similar and lose uniqueness, leading to over-smoothing [19, 20]. Additionally, GNN-based message passing mechanisms, which rely solely on edges to integrate graph structures and node attributes, struggle to fully model user representations due to complex topological structures and relationships [21].

To effectively combine social network and user-item interaction network information, DiffNet++[13] optimizes user and item representations by leveraging higher-order information and uses an attention mechanism to differentiate the importance of various neighbors. MHCN [22] aggregates multiple user embeddings learned through different channels to capture comprehensive higher-order relational information. These GNN-based models effectively mine social network information and improve recommendation performance. However, they often overlook the instability of social networks. In real-world scenarios, passive social relationships (e.g., colleagues or classmates) may not significantly impact user interaction preferences due to differing tastes [23–25]. Relying on such irrelevant social relationships can harm recommendation performance. DANSER [26] addresses this by using a dual graph attention network to learn the static effect of social homogeneity and the dynamic effect of social influence, adjusting weights based on preference differences, though this increases computational cost. ESRF [27] reconstructs social relationships using an auto-encoder constrained by original interaction and social networks, employing a social attention mechanism to selectively aggregate new neighbor information. However, reconstructing sparse social networks can introduce noise.

To address these issues, we propose a model that Social Recommendation with Graph Neural Networks Integrating Transformer and Self-Supervised Learning (SSLGTSR). Our model first uses three positional encoding methods based on shortest path, node degree, and PageRank to embed different granularities of structural topology information from interaction and social graphs into node features, aiding the Transformer in capturing complex graph topology. By integrating Transformer layers with GCN layers, the model captures global dependencies between graph nodes,

not just direct neighbors, while the GNN layers provide neighborhood information to the Transformer, expanding the GNN's receptive field and mitigating over-smoothing. Additionally, we design a social-aware self-supervised learning task module to fully utilize joint information from social and interaction networks. This module aligns user representations by considering interaction commonalities, integrating contextual signals from both relationships.

The main contributions of this paper are as follows:

1. Introducing Transformer layers after GCN layers to effectively utilize topological information from interaction and social graphs, broadening the GNN's receptive field.

2. Designing a self-supervised learning task to align interactive and social behaviors by adapting user representations based on interaction commonalities, assigning reliability weights to useful social relationships, and achieving cross-view alignment between encoded social and interaction embeddings.

3. Experimental results demonstrate that SSLGTSR significantly improves recommendation accuracy.

2 Related work

In this section, we will discuss the related work of social recommendation based on graph neural networks and some self-supervised learning techniques commonly used in recommendation systems.

2.1 Social Recommendation with Graph Neural Networks

Social recommendation can use social network information to improve the performance of user-item interaction networks [28, 29]. In recent years, GNN-based recommendation methods have developed rapidly, offering powerful tools for graph representation learning. In the context of social recommendation, GNN-based social recommendation methods have developed rapidly. GraphRec [10] is the first dual-graph information aggregation architecture, which integrates the first-order neighborhood features of the user-item interaction graph and the social graph through the gated attention mechanism, but its simple neighborhood average pooling operation fails to distinguish the heterogeneity of social influence. DiffNet++ [13] Based on DiffNet [30], a high-order neighborhood attention mechanism is introduced and a dynamic weight allocation strategy is used to realize the adaptive fusion of multi-hop social influence, but its computational complexity increases exponentially with the increase of the number of hops. KCGN [31] adopts multi-behavior modeling for social recommendation, and uses relation-aware GNN to capture social relationships with various behavior dependencies. The RobustSR [32] introduces a user representation learning framework that combines social regularization and multi-view contras learning mechanisms to learn social relation importance weights and enforce discriminatory user representations. However, its noise detection module relies on a preset threshold, which limits its adaptability to dynamic social scenarios. MCLSGI [33] proposes a multi-view contrastive learning method combined with social group influence. This method uses a graph walk strategy to identify a user's potential social groups, and uses a GNN-based framework

to model user preferences from different views to capture the influence of direct and higher-order neighbors and social groups.

In summary, while these GNN-based models have advanced the field of social recommendation, they predominantly rely on local message-passing mechanisms. This inherent design choice limits their ability to capture complex, long-range dependencies across the graph and makes them susceptible to the over-smoothing problem in deeper architectures. By integrating a Graph Transformer, SSLGTSR can explicitly model global context, and by alternating it with GNN layers, it effectively expands the receptive field while mitigating over-smoothing, leading to more powerful and robust user representations.

2.2 Self-supervised learning

Recently, data augmentation based on self-supervised learning (SSL) has emerged as a promising approach to ensure that augmented views have similar representations to the original views [34, 35]. This technique is widely used to learn robust and discriminative embedding representations in recommendation systems [36, 37]. A key SSL paradigm is data augmentation based on contrastive learning, which aligns semantically relevant instances with sampled positive examples while pushing irrelevant samples away as negative examples. However, many systems only contain a single type of interaction data. For example, inspired by graph self-supervised learning, SGL [38] generates contrasting views through random node and edge dropout operations, achieving good performance, but this heuristic augmentation may disrupt crucial graph structures and is not specifically designed for the dual-graph nature of social recommendation. SimGCL [39] enhances SGL's data augmentation method by directly and randomly perturbing user/item embeddings to generate positive samples for contrastive learning, though the random nature of these perturbations does not guarantee the preservation of semantic information. To improve relational learning in social recommendation, S2-MHCN [22] designes an SSL task to capture high-order connections using mutual information maximization. However, it does not explicitly address the semantic gap between the interaction and social domains. DcRec [40] introduces a disentangled contrastive learning framework, but the complexity of disentangling latent factors may not fully capture the complementary signals between user preferences and social influence. CLSR [41] utilizes the social network to generate contrastive samples, yet this approach primarily serves as a data augmentation strategy rather than directly aligning the representations learned from the two distinct views.

In essence, existing SSL approaches in recommendation either rely on heuristic data augmentations that may not preserve essential semantics, or utilize social information merely as a source for generating training samples. They often fall short of explicitly modeling and aligning the distinct semantic information from the interaction view and the social view. To address this gap, we propose a novel cross-view self-supervised learning paradigm. Instead of simple augmentation, SSLGTSR directly enforces consistency between a user's representation learned from their interactions and that learned from their social context. This alignment ensures that social awareness is meaningfully integrated into user preference modeling, resulting in more accurate and coherent recommendations.

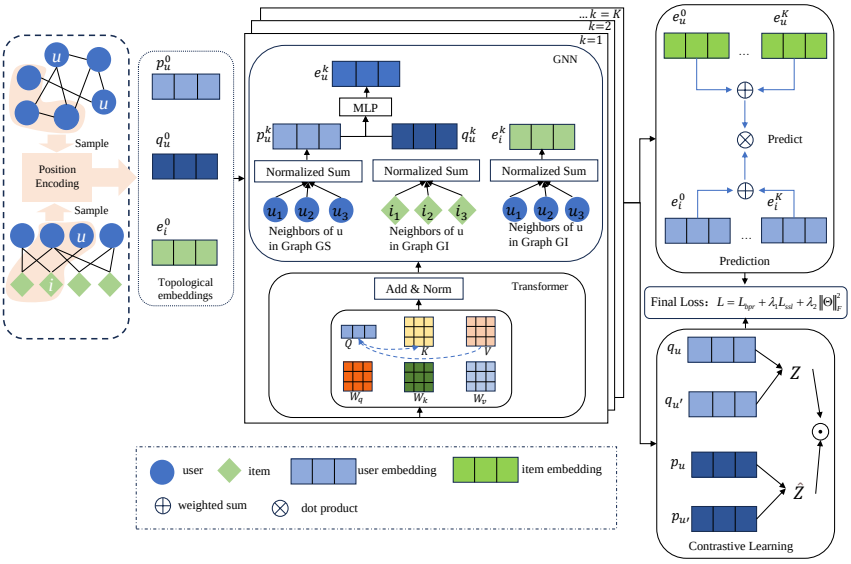


Fig. 1: The architecture of SSLGTSR model

3 Model framework

This chapter provides a detailed introduction to the SSLGTSR model framework, which is divided into five main parts: Attention Sampling; Positional Encoding; Message Propagation; Self-Supervised Learning; Multi-Task Model Optimization. The overall framework of the model is illustrated in Fig. 1.

3.1 Problem definition

Suppose $U = \{u_1, u_2, ..., u_m\}$ and $I = \{i_1, i_2, ..., i_n\}$ are the sets of users and items in the recommender system, respectively, where m is the number of users and n is the number of items. The interaction behaviors between users and items and the social relationships between users are usually represented by user-item interaction graphs and social graphs. Define the user-item interaction graph $GI = \{(u, y_{ui}, i) | u \in U, i \in I\}$, where an edge $y_{ui} = 1$ indicates an interaction between user u and item i , and $y_{ui} = 0$ otherwise. Similarly, define the social graph $GS = \{(u_i, s_{ij}, u_j) | u_i, u_j \in U\}$, where an edge $s_{ij} = 1$ indicates a social relationship between users u_i and u_j , and $s_{ij} = 0$ otherwise. Based on these inputs, the social recommendation task is described as follows: given a user-item interaction graph GI and a social graph GS , predict the interaction probabilities between user u and all items i . Sort these probabilities in descending order and select the top N items to generate a recommendation list for user u .

3.2 Graph attention sampling

Since interaction graphs and social graphs may contain unstable connections, directly aggregating information from all interaction edges can compromise accurate user representation. Considering only the most relevant nodes can reduce computational complexity and filter out irrelevant information from noisy nodes. Therefore, we first compute the user embedding similarity matrix using the following equation:

$$\begin{aligned} \mathbf{S}_r &= \mathbf{E}_r \mathbf{E}_r^T, \\ \mathbf{S}_s &= \mathbf{E}_s \mathbf{E}_s^T \end{aligned} \tag{1}$$

where $\mathbf{E}_r, \mathbf{E}_s \in \mathbb{R}^{m \times d}$ represent the embeddings of user nodes in the interaction graph and social graph, respectively. After the message passing mechanism, each pair of nodes is also influenced by their neighbor nodes, so the preferences of neighbor nodes should be considered before sampling. We update the similarity matrix to include neighbor node preferences using the following equation:

$$\begin{aligned} \mathbf{S}_r &= \mathbf{S}_r + \alpha(\mathbf{G}_I + \mathbf{I})\mathbf{S}_r, \\ \mathbf{S}_s &= \mathbf{S}_s + \alpha(\mathbf{G}_S + \mathbf{I})\mathbf{S}_s \end{aligned} \tag{2}$$

where $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, $\mathbf{G}_I$ and $\mathbf{G}_S$ are adjacency matrices, $\mathbf{I}$ is the identity matrix, and α is the balance factor set to 0.5. Given an input graph $\mathbf{G}$ and its similarity matrix $\mathbf{S}$, we sample the most relevant nodes as attention samples. For a user node $\mathbf{v}_i$, the set of attention samples is defined as follows:

$$Smp(v_i) = top_t(\mathbf{S}_i, *) \tag{3}$$

where $\mathbf{S}_i, *$ represents the data elements of the i-th row of $\mathbf{S}$, and $top_t(\cdot)$ represents selecting the first t largest elements as the attention nodes.

3.3 Global topology information injection

The model uses three common positional encodings to capture the relative position information of nodes in the graph, thereby injecting global topology information to compensate for the lack of position information in the self-attention mechanism. These positional encodings enhance the model's awareness of the graph structure, enabling more accurate representation learning based not only on the node features but also on the relative positions of nodes in the graph. The three common positional encodings introduced into the model are as follows:

Shortest path based Positional Encoding: This encoding captures the relative positional relationship between nodes in the graph by considering the shortest path distances between them.

Degree-Based Positional Encoding: This encoding helps the model capture the local connection information of nodes by considering the degree (number of connections) of each node.

PageRank-Based Positional Encoding: This encoding captures the global importance of nodes within the entire graph by utilizing the PageRank algorithm, helping the model understand the overall structure of the graph.

3.3.1 Shortest-path based positional encoding

The shortest path positional encoding can effectively capture both local and global relationships between users, as well as between users and items. This encoding helps the model understand the community or group that users belong to within the social network and the relevance between users and items. Let $\mathbf{D}$ be the shortest path matrix. For each node $\mathbf{v}_i \in \mathbf{V}$ and its attention sample nodes $\mathbf{v}_j \in Smp(\mathbf{v}_i)$, the shortest path distance is $\mathbf{D}(i,j)$. The shortest path position of each attention sample node $\mathbf{v}_j$ is encoded as follows:

$$PE_{shortest}(v_i, v_j) = MLP(\mathbf{D}(i,j)) \tag{4}$$

where $MLP(\cdot)$ represents a two-layer neural network. This positional encoding integrates the shortest path information, allowing the model to better understand the structural context within the graph.

3.3.2 Degree based positional encoding

Degree encoding can reflect the connectivity and influence of users and items. Users with a high degree may be active users or users with significant influence, while items with a high degree may be popular items. In user social graphs, degree encoding can help quickly identify key users, who may be important targets for recommendations. In the user-item interaction graph, items with a high degree can be recommended preferentially to new users. To obtain the degree vector $\mathbf{d}$, where $\mathbf{d}[i]$ represents the degree of node $\mathbf{v}_i$:

$$PE_{\deg ree}(\mathbf{v}_i) = MLP(\mathbf{d}(\mathbf{v}_i)) \tag{5}$$

this degree vector can then be used for degree-based positional encoding, helping the model to identify key nodes and incorporate connectivity information into the learning process.

3.3.3 PageRank based positional encoding

PageRank encoding provides a global perspective to measure the importance and influence of users and items within the entire graph. Users or items with high PageRank values are more significant and thus more worthy of attention in recommendations. In the user social graph, PageRank encoding can identify users with global influence in the social network. In the user-item interaction graph, it can help identify globally popular items. To get the PageRank vector $\mathbf{PR}$, where $\mathbf{PR}[i]$ represents the PageRank value of node $\mathbf{v}_i$:

$$PE_{PageRank}(\mathbf{v}_i) = MLP(\mathbf{PR}(\mathbf{v}_i)) \tag{6}$$

after obtaining the three positional encodings (shortest path, degree, and PageRank), they are aggregated as follows:

$$\begin{aligned}\mathbf{pe}_i &= MLP(AGG(\mathbf{e}_i^0, PE_{shortest}(\mathbf{v}_i, \mathbf{v}_i), PE_{\deg ree}(\mathbf{v}_i), PE_{PageRank}(\mathbf{v}_i)),\\ \mathbf{pe}_j &= MLP(AGG(\mathbf{e}_j^0, PE_{shortest}(\mathbf{v}_i, \mathbf{v}_j), PE_{\deg ree}(\mathbf{v}_j), PE_{PageRank}(\mathbf{v}_j))\end{aligned} \tag{7}$$

where $\mathbf{e}_i^0$ and $\mathbf{e}_j^0$ are the initial embeddings of nodes $\mathbf{v}_i$ and $\mathbf{v}_j$ respectively, and $AGG(\cdot)$ is the aggregation function, which uses vector concatenation to combine these positional encodings into a comprehensive representation. This aggregated positional encoding enrichments the model's understanding of the local and global graph structure, and this enhanced representation is then used as the input of the first message propagation block, thus injecting structural information into the subsequent representation learning process and enhancing its ability to make accurate recommendations.

3.4 Message propagation

The Transformer layer performs well in handling remote interactions and global information through the self-attention mechanism and positional encoding, and can effectively capture complex dependencies between nodes without being limited by a fixed receptive field. The GNN layer handles local information propagation and aggregation of node features by iteratively updating the representation of each node; therefore, we combine these two modules to obtain the user representation. The model combines a first Transformer layer and a later GNN layer into a basic propagation block, and then stacks K such blocks. The model learns the user representation by stacking K such blocks.

3.4.1 Transformer layer

To reduce computational complexity, the Transformer layer only considers the most relevant samples for each central node. We demonstrate the Transformer layer using a user node u_i in the social graph and its attention sample $Smp(u_i)$, with user and item nodes in the interaction graph being computed similarly.

The input to the Transformer layer is the social embedding matrix of the user after positional encoding, denoted $\mathbf{P} \in \mathbb{R}^{N \times d_{in}}$. The representation of user u_i is denoted as $\mathbf{p}_i$, the attention sample matrix $Smp(u_i)$ is denoted as $\mathbf{P}_i^{Smp} \in \mathbb{R}^{k \times d_{in}}$, and the information is aggregated according to the attention distribution as follows:

$$\mathbf{p}_i = soft\max\left(\frac{\mathbf{QK}^T}{\sqrt{out}}\right)\mathbf{V} \tag{8}$$

where

$$\begin{cases}\mathbf{Q} = \mathbf{p}_i\mathbf{W}_q,\\ \mathbf{K} = \mathbf{P}_i^{Smp}\mathbf{W}_k,\\ \mathbf{V} = \mathbf{P}_i^{Smp}\mathbf{W}_v\end{cases} \tag{9}$$

three linear transformation matrices $\mathbf{W}_q$,$\mathbf{W}_k$,$\mathbf{W}_v \in \mathbb{R}^{d_{in} \times d_{out}}$ are used to project the corresponding representations to the Query, Key, and Value matrices, extending to multi-head attention:

$$Multi - Head(\mathbf{p}_i) = Concat(head_1, \ldots, head_h)\mathbf{W}_h \tag{10}$$

where h is the number of attention heads, $Concat(\cdot)$ is the concatenation function, and $\mathbf{W}_h$ is the projection matrix. After obtaining the multi-head attention, residual connection and layer normalization are performed:

$$\mathbf{p}_i = LayerNorm(\mathbf{p}_i + MultiHead(\mathbf{p}_i)) \tag{11}$$

This process integrates the positional encodings and attention mechanisms, allowing the Transformer layer to effectively capture and aggregate relevant information from both local and distant nodes in the graph.

3.4.2 GNN layer

Based on the user and item embeddings generated by the Transformer layer, graph convolution operations are performed on the user-item interaction graph and the user and item feature embeddings in the social graph. This light graph convolution simplifies the traditional GCN by eliminating linear transformations and nonlinear activation operations [8]. In the kth propagation layer, the neighborhood information embeddings in the (k-1) propagation layers are aggregated (the 0th propagation layer embeddings are the initial embeddings), iterated K times, and the updated embeddings are output.

For user u , the user embedding $\mathbf{e}_u^k$ output from layer k is updated using the interaction graph GI and the social graph GS . We generate two embeddings, denoted $\mathbf{q}_u^k$ and $\mathbf{p}_u^k$, for user u in the user-item interaction graph and the social graph, respectively:

$$\mathbf{q}_u^k = \sum_{i \in N_u^{GI}} \frac{1}{\sqrt{|N_u^{GI}|}\sqrt{|N_i^{GI}|}} \mathbf{q}_i^{k-1} \tag{12}$$

$$\mathbf{p}_u^k = \sum_{v \in N_u^{GS}} \frac{1}{\sqrt{|N_u^{GS}|}\sqrt{|N_v^{GS}|}} \mathbf{p}_u^{k-1} \tag{13}$$

where N_u^{GI} denotes the set of neighbors of user u in the interaction graph and N_u^{GS} denotes the set of neighbors of user u in the social graph. For an embedding $\mathbf{e}_i^k$ of item i output from layer k , it propagates in the user-item interaction graph as follows:

$$\mathbf{e}_i^k = \sum_{u \in N_i^{GI}} \frac{1}{\sqrt{|N_i^{GI}|}\sqrt{|N_u^{GI}|}} \mathbf{e}_u^{k-1} \tag{14}$$

after the K-layer convolution operation, the node embeddings of each layer are combined to form the final embedding combination formula as follows:

$$\begin{cases} \mathbf{p}_u = \sum_{k=0}^{K} \alpha_k \mathbf{p}_u^k, \\ \mathbf{q}_u = \sum_{k=0}^{K} \alpha_k \mathbf{q}_u^k, \\ \tilde{\mathbf{e}}_u = \mathbf{W}_6(\sigma(\mathbf{W}_4 \mathbf{q}_u) \| \sigma(\mathbf{W}_5 \mathbf{p}_u)), \\ \mathbf{e}_u = \frac{\tilde{\mathbf{e}}_u}{\|\tilde{\mathbf{e}}_u\|_2}, \\ \mathbf{e}_i = \sum_{k=0}^{K} \alpha_k \mathbf{e}_i^k \end{cases} \tag{15}$$

where the aggregation factor α_k is uniformly set to $1/k+1$, σ is the tanh activation function, and $\mathbf{W}_4, \mathbf{W}_5 \in \mathbb{R}^{d \times d}$ and $\mathbf{W}_6 \in \mathbb{R}^{d \times 2d}$ are trainable matrices.

This approach leverages the complementary strengths of the Transformer and GNN layers to capture both global dependencies and local structures, enhancing the overall user and item representations in the recommendation system.

3.5 Cross-view self-supervised learning

To ensure that the user-item relationships and user dependencies learned by the model complement each other, we design a cross-view self-supervised learning paradigm that integrates these two relational context signals. In this self-supervised learning module, we combine an adaptive cross-view alignment task with auxiliary self-supervised signals to enhance the main learning task. In general, the interaction similarity $z_{i,i'} = [\mathbf{q}_i, \mathbf{q}_{i'}]$ between a user pair (i, i') , similarly social similarity $\hat{z}_{i,i'} = [\mathbf{p}_i, \mathbf{p}_{i'}]$. To bridge the semantic gap between interactive and social views, we introduce a learnable similarity function to map interaction semantics into a latent embedding space for cross-view alignment as follows.

$$z_{i,i'} = sigm(\mathbf{d}^T \cdot \sigma(\mathbf{T} \cdot [\mathbf{q}_i, \mathbf{q}_{i'}] + \mathbf{q}_i + \mathbf{q}_{i'} + \mathbf{c})) \tag{16}$$

where $sigm(\cdot)$ denotes the Sigmoid activation function, $\sigma(\cdot)$ denotes the LeakyReLU activation function, and $\mathbf{d} \in \mathbb{R}^d, \mathbf{T} \in \mathbb{R}^{d \times 2d}, \mathbf{c} \in \mathbb{R}^d$ are learnable parameters. we then propose a self-supervised learning task that incorporates social awareness. The cross-view alignment loss function is defined as:

$$L_{ssl} = \sum_{(u_i, u_{i'})} \max(0, 1 - z_{i,i'} \hat{z}_{i,i'}) \tag{17}$$

where the user pairs $(u_i, u_{i'})$ are randomly sampled from the user set U .

This alignment task ensures that the model learns consistent user representations across both interaction and social views, improving the overall recommendation performance by leveraging the complementary information from both types of graphs.

3.6 Multi-task Model Optimization

The learning process of SSLGTSR involves multi-task training for model optimization, which includes both recommendation optimization loss and self-supervised learning tasks. The process is designed to ensure that the model not only provides accurate recommendations but also learns robust and discriminative embeddings through self-supervised learning. First, we compute the user's preference for items using the inner product of the final embeddings of users and items:

$$\hat{r}_{ui} = \mathbf{e}_u^T \mathbf{e}_i \tag{18}$$

the recommendation optimization loss model employs the Bayesian Personalized Ranking (BPR) loss, which is commonly used in graph convolutional network recommendation algorithms:

$$L_{bpr} = \sum_{(u,i^+,i^-)\in O} -\ln \phi(\widehat{r}_{ui^+} - \widehat{r}_{ui^-}) \tag{19}$$

where $O = \{(u, i^+, i^-) \,|(u, i^+) \in R^+, (u, i^-) \in R^-\}$ represents the training set, R^+ is the observed interaction between user u and item i^+ in the training dataset, R^- is the set of sampled unobserved interactions. By combining the self-supervised learning objective with the BPR loss, the joint optimization loss is defined as follows:

$$L = L_{bpr} + \lambda_1 L_{ssl} + \lambda_2 \|\Theta\|_F^2 \tag{20}$$

where λ_1 and λ_2 are hyperparameters used for balancing the losses, and represents all parameters of the model.

4 Experiment analysis

In this section, we will evaluate the effectiveness of our proposed method through detailed experiments and compare it with other state-of-the-art methods on four public benchmark datasets. Furthermore, we will conduct ablation experiments to demonstrate the effectiveness of some components in our framework. After that, we further investigate the effect of parameters in SSLGTSR and visualize the attention weights.

4.1 Experimental settings

4.1.1 Datasets

To verify the effectiveness of the SSLGTSR model, experiments were conducted on three public datasets: LastFM, Ciao and Douban-Book [3]. The statistical data for

Table 1: Statistics of three datasets

Dataset	#User	#Item	#Interaction	Density(Interaction)	#Relation
LastFM	1892	17632	92834	0.278%	25434
Ciao	7375	105114	284086	0.037%	57544
Douban-Book	13024	22347	792062	0.272%	169150

these datasets are shown in Table 1. Each dataset was split into training, validation, and test sets with a 7:2:1 ratio, ensuring independent and non-overlapping splits.

4.1.2 Parameter settings and evaluation metrics

The experiments were carried out on a server platform equipped with a Xeon(R) Platinum 8358P processor and an NVIDIA GeForce RTX 3090 graphics card (24GB). The system used CUDA version 11.3, Python version 3.8.3, and PyTorch version 1.11.0. The Adam optimizer, with a learning rate of 0.001, was employed to optimize the prediction model and update the model parameters. An early stopping strategy was used to halt the training process when performance ceased to improve. By default, the number of attention samples was set to 15, the number of attention heads was 2, the dimension of the node embedding vector was 64, and the batch size was 1024. User and item embeddings were initialized using a Gaussian distribution with a mean of 0 and a standard deviation of 0.01. The number of graph convolutional layers K was set to 4 by default, the self-supervised learning loss $\lambda_1 = 1e^{-5}$, and the regularization coefficient $\lambda_2 = 1e^{-5}$.

The performance of the proposed SSLGTSR model was evaluated using two widely used metrics for Top-N ranking recommendations: Recall (Recall@N) and Normalized Discounted Cumulative Gain (NDCG@N), with N values set to 10 and 20.

4.1.3 Baseline

The performance of SSLGTSR was compared with eight recent social recommendation baseline models. The hyperparameter settings in the original models were kept consistent with those providing the best performance, except where specifically mentioned:

SBPR [5]: A recommendation model based on matrix factorization, utilizing user social relationships to enhance the accuracy of personalized ranking.

LightGCN [9]: A simplified graph convolutional network using only linear propagation and aggregation between neighbors for embedding calculations.

DiffNet++ [13]: Extends DiffNet by employing a graph attention network for neighbor aggregation on social and interaction graphs to generate user and item representations.

S2-MHCN [22]: Utilizes self-supervised learning and multi-channel hypergraph convolutional networks to model complex user relationships for graph-based social recommendations.

SimGCL [39]: Proposes a simple and effective contrastive learning method for recommendations without graph augmentation.

SocialLGN [18]: Designs a lightweight graph convolutional network-based representation propagation mechanism for user-item interaction and social graphs, making it more suitable for social recommendation problems.

GFormer [21]: Combines generative self-supervised learning with a graph Transformer architecture to learn appropriate interaction patterns as self-supervised signals under task-adaptive collaboration principle discovery.

DSL [37]: Achieves denoising cross-view alignment by comparing social network user representations with global user representations for similarity between different users.

4.2 Recommendation performance evaluation

The performance of the baseline models and SSLGTSR on the three datasets is presented in Table 2. The best performance results are highlighted in bold, while the second-best results are underlined.

Table 2: Comparison of SSLGTSR and baseline models on three datasets

dataset	metric	SBPR	LightGCN	DiffNet++	S2-MHCN	SimGCL	Social LGN	GFormer	DSL	SSLGTSR	Imp
LastFM	Recall@10	0.1442	0.1951	0.1875	0.2037	0.2005	0.2026	0.2029	0.1995	**0.2087**	2.86%
	Recall@20	0.2070	0.2769	0.2651	0.2802	0.2808	0.2794	0.2813	0.2823	**0.2889**	2.34%
	NDCG@10	0.1749	0.2340	0.2233	0.2440	0.2486	0.2566	0.2534	0.2383	**0.2667**	3.94%
	NDCG@20	0.1978	0.2788	0.2665	0.2841	0.2847	0.2883	0.2884	0.2812	**0.2965**	2.81%
Ciao	Recall@10	0.0259	0.0410	0.0357	0.0433	0.0375	0.0430	0.0433	0.0356	**0.0445**	2.77%
	Recall@20	0.0412	0.0591	0.0565	0.0611	0.0504	0.0618	0.0623	0.0560	**0.0650**	4.33%
	NDCG@10	0.0266	0.0437	0.0371	0.0422	0.0376	0.0441	0.0431	0.0436	**0.0456**	3.40%
	NDCG@20	0.0307	0.0478	0.0442	0.0489	0.0409	0.0486	0.0455	0.0405	**0.0504**	3.70%
Douban Book	Recall@10	0.0887	0.0900	0.0917	0.0983	0.0977	0.1001	0.0989	0.1005	**0.1052**	4.68%
	Recall@20	0.1134	0.1262	0.1171	0.1441	0.1478	0.1511	0.1519	0.1503	**0.1580**	4.02%
	NDCG@10	0.1152	0.1306	0.1265	0.1214	0.1307	0.1315	0.1303	0.1317	**0.1378**	4.56%
	NDCG@20	0.1226	0.1302	0.1214	0.1316	0.1343	0.1361	0.1369	0.1376	**0.1413**	2.69%

The results in Table 2 demonstrate that the SBPR model, based on matrix factorization, underperforms compared to GNN models, highlighting the latter's capability to learn complex social relationships and high-order features more effectively. LightGCN outperforms DiffNet++ by removing linear transformation and nonlinear activation operations, which negatively impact recommendation performance. SocialLGN further improves on LightGCN by incorporating social information and considering higher-order graph structures. Self-supervised learning models such as S2-MHCN, SimGCL, and DSL significantly outperform LightGCN across all datasets and metrics, effectively integrating social relationships into recommendations by learning more discriminative user representations. GFormer, which employs Graph Transformer to learn context-invariant user preference information, performs better than other self-supervised learning (SSL) models on most indicators, mitigating the noise from observed behavior data that may contain biases and confounding factors. SSLGTSR

achieves the best performance across all metrics on each dataset, with an average increase of 2.60% in Recall@N and 3.38% in NDCG@N on LastFM, 3.55% in both metrics on Ciao, and 4.35% in Recall@N and 3.63% in NDCG@N on Douban-Book. This superior performance indicates that SSLGTSR effectively leverages Graph Transformer to inject global topology information into social and interaction graphs through three positional encodings, enhancing global relation learning. The adaptive cross-view alignment task in SSLGTSR, combined with an auxiliary self-supervised signal, aligns the similarity of user-user pairs in the interaction domain with that in the social domain, ensuring that the learned user-item relationships and user dependencies complement each other. Overall, SSLGTSR demonstrates its effectiveness in leveraging graph structure learning and self-supervised learning to achieve state-of-the-art performance in social recommendation tasks.

4.3 Hyperparameter sensitivity analysis

In this section, we examine the impact of three critical hyperparameters on the performance of SSLGTSR: the number of attention samples t , the number of message propagation layers K , and the SSL loss weight.

4.3.1 Influence of the number of attention samples

To investigate the effect of the number of attention samples, we set t to 5, 10, 15, 20, 25, 30 and measure the performance in terms of Recall@10 and NDCG@10 on the LastFM, Ciao, and Douban-Book datasets. The results are illustrated in Fig. 2.

According to the results in Fig. 2, as t increases from 5 to 30, the performance of SSLGTSR initially improves and then either plateaus or slightly declines. When t is small, the attention mechanism samples too few nodes, limiting the model's ability to capture the topological information of the interaction and social graphs. As t increases, the model's performance improves due to a broader receptive field, which enhances the model's capability. However, a very large receptive field may introduce noise, impairing performance, and excessive node sampling can also lead to increased computational overhead.

4.3.2 Influence of the number of message propagation layers

To study the influence of the number of message propagation layers K , we set the number of graph convolution layers to 1, 2, 3, 4, 5, 6 and compare the results with the performance of the SocialLGN model. Since the trends in the results are consistent across all three datasets, we present the results for the Douban-Book dataset in Fig. 3.

As depicted in Fig. 3, a smaller K restricts GNN nodes from aggregating higher-order neighbor information. When K=0, the performance of graph neural networks degrades significantly. Conversely, stacking too many layers can lead to over-smoothing, resulting in indistinguishable node embeddings and diminished model performance. SocialLGN achieves optimal performance at K =3, but beyond this point, it suffers from over-smoothing. In contrast, SSLGTSR, which embeds the Transformer layer within the GNN layer for multiple iterations, benefits from the multi-head

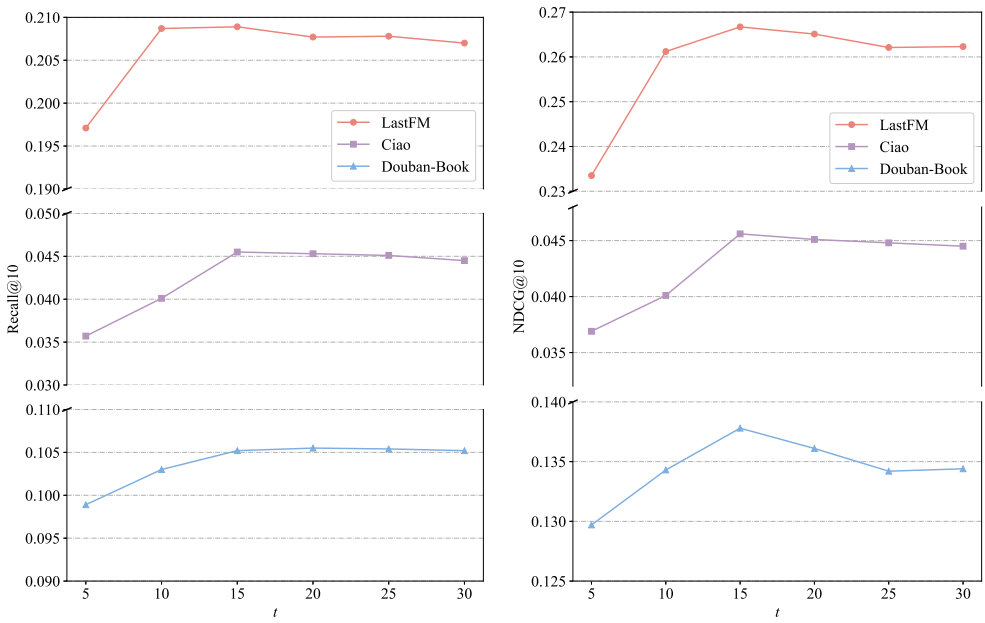


Fig. 2: Sensitivity analysis for t

self-attention mechanism. This mechanism enables the model to adaptively aggregate global information, effectively mitigating the over-smoothing issue common in traditional graph neural networks.

4.3.3 Influence of SSL loss weight

The SSL loss controls the regularization strength of self-supervised signals, with an appropriate weight improving model learning on highly skewed distributed data. To study the impact of the SSL loss weight λ_2 on model performance, we set λ_2 to 1e-6, 1e-5, 1e-4, 1e-3 and present the results in Fig. 4.

Fig. 4 shows that the best performance is achieved with $\lambda_2 = 1e-5$ across all three datasets. This suggests that a moderate level of influence from the SSL task is beneficial. At this setting, the model effectively integrates the auxiliary self-supervised signals to enhance the learned representations without detracting from the main recommendation task. If λ_2 is too low, the model might not benefit fully from the self-supervised signals. Conversely, if λ_2 is too high, the model's learning could be overly influenced by the self-supervised task, potentially detracting from the primary objective of accurate recommendation.

4.4 Ablation experiment

To comprehensively validate the rationality and effectiveness of our proposed SSLGTSR model, we conduct two distinct ablation studies in this section. First,

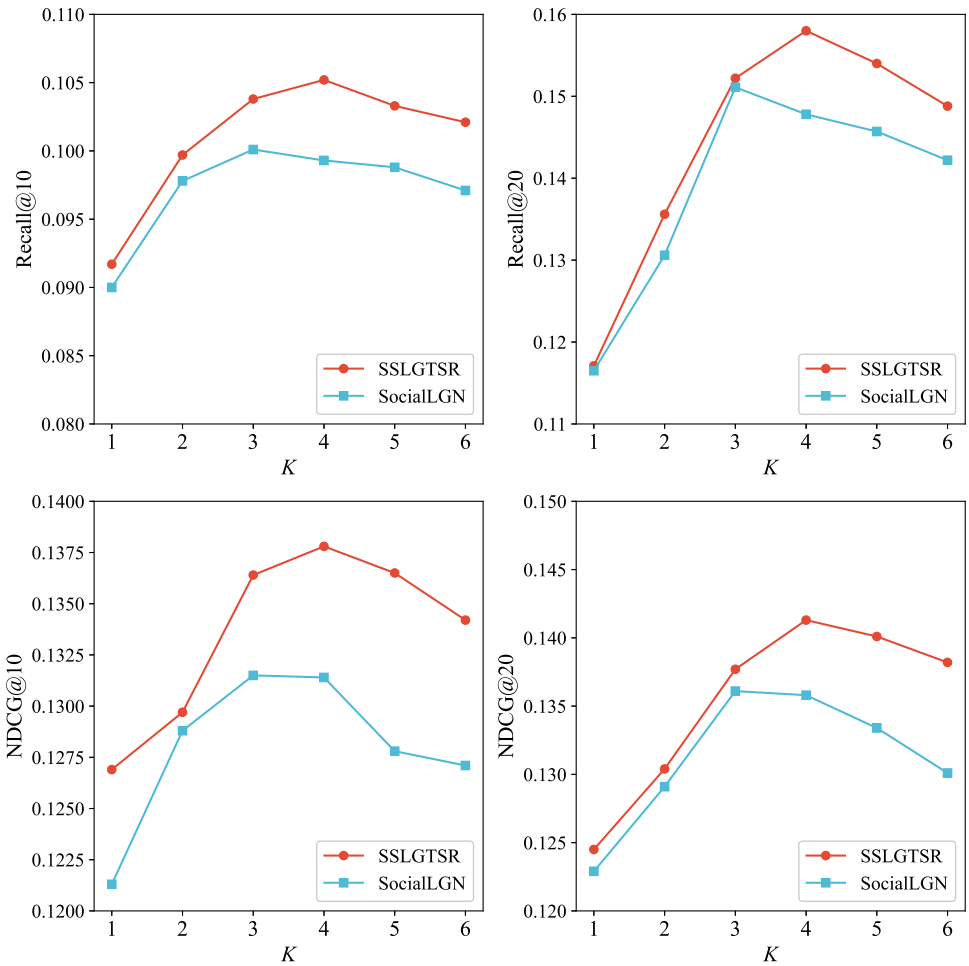


Fig. 3: Sensitivity analysis for K

we will analyze the impact of each core module—namely the global topology information injection, the Transformer layer, and the cross-view self-supervised learning module—to demonstrate their individual contributions to the model's performance. Subsequently, we will investigate the architectural design by comparing different layer stacking orders, thereby providing empirical evidence for our chosen configuration.

4.4.1 Analysis of Model Modules

To verify the rationality of the SSLGTSR model design, an ablation study is performed to investigate the effectiveness of each module separately. When global topology information injection, the Transformer layer, and cross-view self-supervised learning are

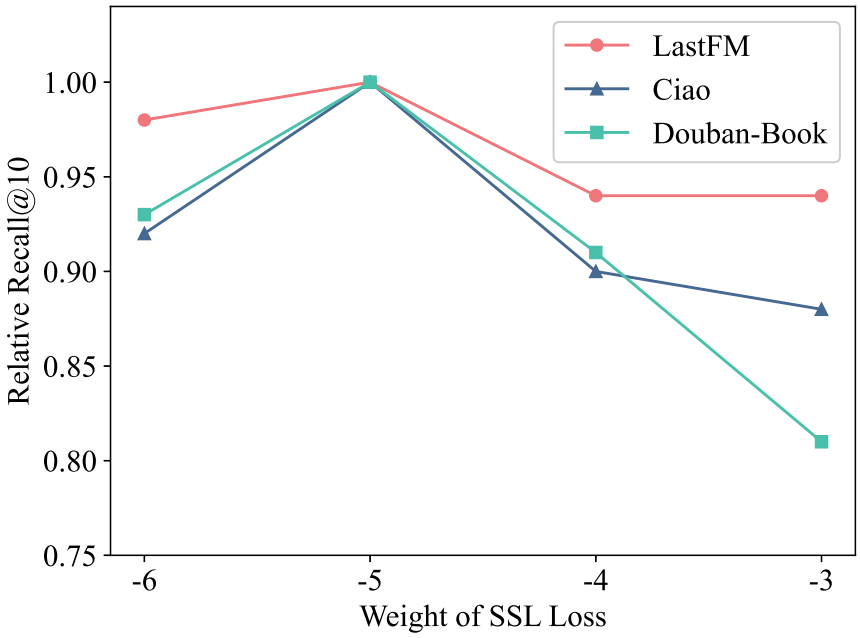


Fig. 4: Sensitivity analysis for λ_2

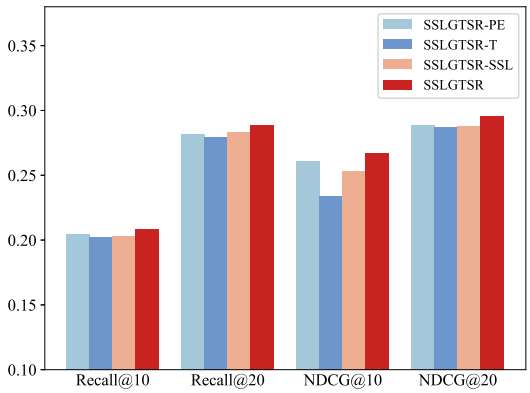
removed from SSLGTSR, the resulting variant models are denoted as SSLGTSR-PE, SSLGTSR-T, and SSLGTSR-SSL, respectively. SSLGTSR is the full version of the proposed model. The ablation study was conducted on three datasets, and the experimental results are shown in Fig. 5.

From Fig. 5, it is evident that the removal of any module results in a degradation of model performance, indicating that all the proposed modules in SSLGTSR are effective. Specifically:

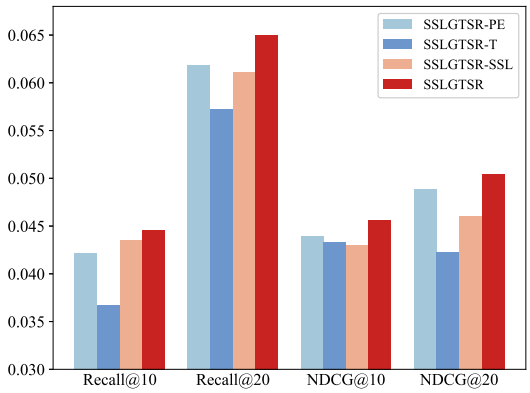
SSLGTSR-PE: This variant removes the global topology information injection module, which weakens the model's ability to perceive the global topology of the social graph and interaction graph, thereby affecting the model's performance. The global topology information helps the model understand the broader structural relationships, enhancing its recommendation capabilities.

SSLGTSR-T: By removing the Transformer layer in the message propagation module, this variant reduces the model's complexity and the number of parameters. However, this simplification limits the node's receptive field to only its direct neighbors, preventing the model from fully understanding the semantic relationships and handling remote interactions and global information. The Transformer layer is crucial for capturing long-range dependencies and rich contextual information.

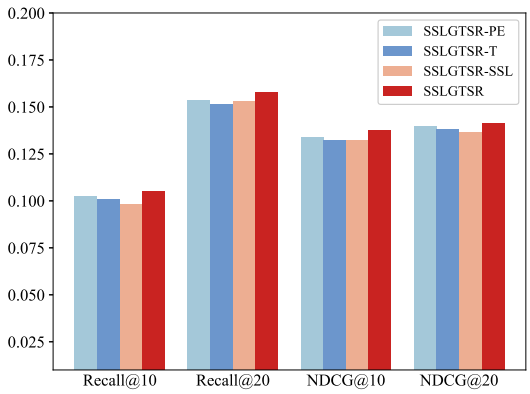
SSLGTSR-SSL: This variant eliminates the self-supervised learning module, preventing the model from realizing the dependency between user nodes in the user graph and the social graph. Without self-supervised learning, the model loses the benefit



(a) LastFM



(b) Ciao



(c) Douban-Book

Fig. 5: Ablation experiments for SSLGTSR on three datasets

Table 3: Performance comparison of different layer architectures on three datasets.

Dataset	Model	Recall@10	Recall@20	NDCG@10	NDCG@20
LastFM	SSLGTSR (Proposed)	**0.2087**	**0.2889**	**0.2667**	**0.2965**
	SSLGTSR-GNN-First	0.2032	0.2835	0.2603	0.2922
	SSLGTSR-Separated	0.1977	0.2644	0.2431	0.2771
Ciao	SSLGTSR (Proposed)	**0.0445**	**0.0650**	**0.0456**	**0.0504**
	SSLGTSR-GNN-First	0.0433	0.0625	0.0439	0.0488
	SSLGTSR-Separated	0.0415	0.0587	0.0398	0.0454
Douban-Book	SSLGTSR (Proposed)	**0.1052**	**0.1580**	**0.1378**	**0.1413**
	SSLGTSR-GNN-First	0.1015	0.1533	0.1341	0.1389
	SSLGTSR-Separated	0.0987	0.1477	0.1301	0.1325

of leveraging auxiliary tasks to enhance representation learning, leading to reduced performance.

The experimental results demonstrate the importance of each module. The global topology information injection, Transformer layer, and self-supervised learning module all contribute significantly to the overall performance of SSLGTSR.

4.4.2 Analysis of Layer Stacking Order

A key architectural choice in our SSLGTSR model is the alternating integration of Transformer and GNN layers. To validate our design and understand the impact of this specific arrangement, we conduct an ablation study comparing our proposed stacking order with several alternative architectures. This analysis aims to empirically demonstrate the rationale behind placing the Transformer layer before the GNN layer within each propagation block.

We compare the following four architectural variants:

SSLGTSR: This is our proposed architecture, where each propagation block consists of a Transformer layer followed by a GNN layer. The hypothesis is that capturing global dependencies first provides richer, context-aware representations that can then be more effectively aggregated locally by the GNN.

SSLGTSR-GNN-First: In this variant, we reverse the order within each block. The model first aggregates local neighborhood information and then uses the Transformer to model global relationships among these locally-smoothed features.

SSLGTSR-Separated: This variant adopts a non-alternating, sequential stacking approach. All Transformer layers are applied first, followed by all GNN layers. This is a common strategy for combining different types of network layers.

To ensure a fair comparison, all other hyperparameters were kept consistent with our main experimental settings. The performance of these variants on the three datasets is presented in Table 3.

The results in Table 3 clearly demonstrate that our proposed SSLGTSR (Transformer-First) architecture consistently outperforms all other variants on all datasets and metrics. We analyze these findings as follows.

Both alternating architectures perform better than the separated stacking variants. This suggests that the iterative interplay between local and global information propagation is crucial. In a separated design, stacking multiple GNN layers can lead to significant over-smoothing, causing node representations to become indistinguishable before the Transformer layers have a chance to process them. The alternating structure mitigates this issue by injecting global context at each propagation step, thereby preserving feature diversity.

Within the superior alternating paradigm, our proposed Transformer-First order shows a clear advantage over the GNN-First order. We attribute this to the fact that placing the Transformer first allows the model to capture complex, long-range dependencies at the beginning of each block. The resulting globally-aware node representations then provide a richer and more informative basis for the GNN layer to perform its local neighborhood aggregation. Conversely, the GNN-First approach may prematurely average out unique node features within local communities, providing a less distinctive input for the subsequent Transformer layer and limiting its ability to model nuanced global relationships.

In summary, this analysis validates our architectural choice, demonstrating that the alternating Transformer-first structure is a more effective design for integrating global and local information in graph-based social recommendation, leading to more powerful and robust user and item representations.

5 Conclusion

In this paper, we proposed SSLGTSR, a novel social recommendation model that effectively integrates a Graph Transformer with a GNN and self-supervised learning. By leveraging positional encodings for topological awareness, alternating Transformer-GNN layers to capture both global and local dependencies, and a cross-view self-supervised task for semantic alignment, our model successfully addresses key challenges in the field. Our experiments validate this approach, with SSLGTSR significantly outperforming state-of-the-art baselines on three public datasets, achieving performance gains of up to 4.35% in Recall and 3.63% in NDCG. These findings underscore the significant potential of combining global context-aware architectures with local message-passing for creating more powerful recommendation systems. However, the computational cost of Transformer's self-attention mechanism may pose scalability challenges on extremely large-scale graphs. Moreover, the performance of the model is inherently dependent on the quality and density of the social graph, and sparse or noisy social connections may limit its effectiveness. For future work, we plan to explore graph signal processing from a spectral perspective. By employing graph filters, we can more explicitly decompose signals in the graph frequency domain. This could enable us to selectively amplify signals related to homophily (low-frequency) while filtering out noise or modeling heterophily (high-frequency), leading to more robust and interpretable representations.

Declarations

Some journals require declarations to be submitted in a standardised format. Please check the Instructions for Authors of the journal to which you are submitting to see if you need to complete this section. If yes, your manuscript must contain the following sections under the heading 'Declarations':

- Funding
  The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This work was supported in part by the Natural Science Foundation of Xinjiang Uygur Autonomous Region of China (Grant No. 2023D01C17, 2023D01A123, 2022D01C692), the National Natural Science Foundation of China (Grant Nos. 62262064, 61862060), the Xinjiang Uygur Autonomous Region Science and Technology Plan Project Tianshan Innovation Team Plan (Grant No. 2023D4012).

- Conflict of interest/Competing interests (check journal-specific guidelines for which heading to use)
  The author(s) declare that the authors have no competing interests as defined by Springer, or other interests that might be perceived to influence the results and/or discussion reported in this paper.
- Data availability
  All data for this study are available from public repositories.
- Author contribution
  Y.X. conceived and designed the study. M.S. developed the methodology, conducted the experiments and analyzed the data, and wrote the main manuscript text. L.G. and Q.Z. collected the data and performed part of the experiments. Y.J. reviewed the manuscript and provided critical feedback. All authors reviewed and approved the final manuscript.

References

[1] Marsden PV, Friedkin NE (1993) Network studies of social influence. Sociological Methods & Research 22(1):127–151

[2] McPherson M, Smith-Lovin L, Cook JM (2001) Birds of a feather: Homophily in social networks. Annual review of sociology 27(1):415–444

[3] Li X, Sun L, Ling M, et al (2023) A survey of graph neural network based recommendation in social networks. Neurocomputing 549:126441

[4] Yang L, Liu Z, Dou Y, et al (2021) Consisrec: Enhancing gnn for social recommendation via consistent neighbor aggregation. In: Proceedings of the 44th international ACM SIGIR conference on Research and development in information retrieval, pp 2141–2145

[5] Zhao T, McAuley J, King I (2014) Leveraging social connections to improve personalized ranking for collaborative filtering. In: Proceedings of the 23rd

ACM international conference on conference on information and knowledge management, pp 261–270

[6] Hoang VT, Jeon HJ, You ES, et al (2023) Graph representation learning and its applications: A survey. Sensors 23(8):4168

[7] Chen J, Xin X, Liang X, et al (2022) Gdsrec: Graph-based decentralized collaborative filtering for social recommendation. IEEE transactions on knowledge and data engineering 35(5):4813–4824

[8] Yang XY, Xu F, Yu J, et al (2023) Graph neural network-guided contrastive learning for sequential recommendation. Sensors 23(12):5572

[9] He X, Deng K, Wang X, et al (2020) Lightgcn: Simplifying and powering graph convolution network for recommendation. In: Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, pp 639–648

[10] Fan W, Ma Y, Li Q, et al (2019) Graph neural networks for social recommendation. In: The world wide web conference, pp 417–426

[11] El Alaoui D, Riffi J, Sabri A, et al (2022) Deep graphsage-based recommendation system: jumping knowledge connections with ordinal aggregation network. Neural Computing and Applications 34(14):11679–11690

[12] Gao C, Wang X, He X, et al (2022) Graph neural networks for recommender system. In: Proceedings of the fifteenth ACM international conference on web search and data mining, pp 1623–1625

[13] Wu L, Li J, Sun P, et al (2020) Diffnet++: A neural influence and interest diffusion network for social recommendation. IEEE Transactions on Knowledge and Data Engineering 34(10):4753–4766

[14] El Alaoui D, Riffi J, Sabri A, et al (2024) Comparative study of filtering methods for scientific research article recommendations. Big Data and Cognitive Computing 8(12):190

[15] El Alaoui D, Riffi J, Sabri A, et al (2025) A novel session-based recommendation system using capsule graph neural network. Neural Networks p 107176

[16] El Alaoui D, Riffi J, Sabri A, et al (2024) Contextual recommendations: dynamic graph attention networks with edge adaptation. IEEE Access

[17] Liu Y, Chen L, He X, et al (2020) Modelling high-order social relations for item recommendation. IEEE Transactions on Knowledge and Data Engineering 34(9):4385–4397

[18] Liao J, Zhou W, Luo F, et al (2022) Sociallgn: Light graph convolution network for social recommendation. Information Sciences 589:595–607

[19] Cai X, Huang C, Xia L, et al (2023) Lightgcl: Simple yet effective graph contrastive learning for recommendation. arXiv preprint arXiv:230208191

[20] Chen M, Wei Z, Huang Z, et al (2020) Simple and deep graph convolutional networks. In: International conference on machine learning, PMLR, pp 1725–1735

[21] Li C, Xia L, Ren X, et al (2023) Graph transformer for recommendation. In: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 1680–1689

[22] Yu J, Yin H, Li J, et al (2021) Self-supervised multi-channel hypergraph convolutional network for social recommendation. In: Proceedings of the web conference 2021, pp 413–424

[23] Liu N, Tan Q, Li Y, et al (2019) Is a single vector enough? exploring node polysemy for network embedding. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp 932–940

[24] Epasto A, Perozzi B (2019) Is a single embedding enough? learning node representations that capture multiple social contexts. In: The world wide web conference, pp 394–404

[25] El Alaoui D, Riffi J, Aghoutane B, et al (2021) Overview of the main recommendation approaches for the scientific articles. In: International Conference on Business Intelligence, Springer, pp 107–118

[26] Wu Q, Zhang H, Gao X, et al (2019) Dual graph attention networks for deep latent representation of multifaceted social effects in recommender systems. In: The world wide web conference, pp 2091–2102

[27] Yu J, Yin H, Li J, et al (2020) Enhancing social recommendation with adversarial graph convolutional networks. IEEE Transactions on knowledge and data engineering 34(8):3727–3739

[28] El Alaoui D, Riffi J, Aghoutane B, et al (2021) Collaborative filtering: comparative study between matrix factorization and neural network method. In: Networked Systems: 8th International Conference, NETYS 2020, Marrakech, Morocco, June 3–5, 2020, Proceedings 8, Springer, pp 361–367

[29] El Alaoui D, Riffi J, Sabri A, et al (2024) Social recommendation system based on heterogeneous graph attention networks. International Journal of Data Science and Analytics pp 1–17

[30] Wu L, Sun P, Fu Y, et al (2019) A neural influence diffusion model for social recommendation. In: Proceedings of the 42nd international ACM SIGIR conference on research and development in information retrieval, pp 235–244

[31] Huang C, Xu H, Xu Y, et al (2021) Knowledge-aware coupled graph neural network for social recommendation. In: Proceedings of the AAAI conference on artificial intelligence, pp 4115–4122

[32] Wu B, Kang Y, Guan B, et al (2023) We are not so similar: Alleviating user representation collapse in social recommendation. In: Proceedings of the 2023 ACM International Conference on Multimedia Retrieval, pp 378–387

[33] Zhang C, Li G, Zhang H (2024) Multi-view contrastive learning with virtual social group influence for social recommendation. Knowledge-Based Systems 294:111751

[34] Lee OJ, et al (2024) Transitivity-preserving graph representation learning for bridging local connectivity and role-based similarity. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 12456–12465

[35] Lee OJ, et al (2025) Pre-training graph neural networks on molecules by using subgraph-conditioned graph information bottleneck. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 17204–17213

[36] Yang Y, Huang C, Xia L, et al (2023) Debiased contrastive learning for sequential recommendation. In: Proceedings of the ACM web conference 2023, pp 1063–1073

[37] Wang T, Xia L, Huang C (2023) Denoised self-augmented learning for social recommendation. arXiv preprint arXiv:230512685

[38] Wu J, Wang X, Feng F, et al (2021) Self-supervised graph learning for recommendation. In: Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval, pp 726–735

[39] Yu J, Yin H, Xia X, et al (2022) Are graph augmentations necessary? simple graph contrastive learning for recommendation. In: Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval, pp 1294–1303

[40] Wu J, Fan W, Chen J, et al (2022) Disentangled contrastive learning for social recommendation. In: Proceedings of the 31st ACM International Conference on Information & Knowledge Management, pp 4570–4574

[41] Zhuang J, Meng S, Zhang J, et al (2023) Contrastive learning based graph convolution network for social recommendation. ACM Transactions on Knowledge Discovery from Data 17(8):1–21